```kotlin
fun main() {  new *
    for (i in 1 ≤ .. ≤ 10) {
        println("Hello, welcome to the tutorial")
    }
    TODO( reason: "Write some cool code so that they are impressed")
}
```

Programming Tutorial - 27

2. Tutorial - Control Structures

Kuzey Aydınhan | 14. November 2025

Contents

1. Overview

2. Data Type Operators

3. Branch Structures

If Statements
Switch Cases

4. Loops

For Loops
While Loops
Break & Continue

Information

Important

- Homework 1 deadline: 20.11. **06:00**
- Sign up for Übungsschein in Campus System until 10.12. 12:00
- There is no tutorial on 19.12. (1 week before xmas)
- There will be a Saalübung on 02.02.

Last Week

Objects & Classes

Introduction to object oriented programming(OOP) for software modelling.

- Classes: Blueprints of objects
- Objects: Instances of classes
- Attributes: Properties of objects
- Methods: Methods define how objects act

Last Week

Data Types

Primitive and non-primitive data types.

- `Strings` and `Enums` aren't primitive
- Primitive data types can be extended through classes & `Enums`
- `Enums` are predefined constants
- `Enums` help write readable code

Motivation

- Variable initialization
- Operations with data types
- In example: `boolean isItTrue = !true && false`
- If/Switch cases
- For/While loops

Variable Initialization

- Keyword: `Type name = value`
- Non-primitive types can also be initialized with `null`
- Primitive types can't be initialized with `null`

```java
void example() {
    String name = "A name";
    String nullableName = null;

    // Create reference
    int lateInit;

    // Initialize
    lateinit = 10;

    Boolean nullableVar = null;
    boolean nonNullableVar = null; // This is wrong
}

```

Important

- **`B`**`oolean` is an extension class of primitive **`b`**`oolean` type
- Which means **`B`**`oolean` is non-primitive whereas **`b`**`oolean` is primitive
- This applies for other types like **`I`**`nteger` vs `int` etc.

String Operators

Method	Explanation	Example
`+`	Concatenation	`"ab" + "bc"; // = "abbc"`
`length()`	String length	`"abc".length(); // = 3`
`substring(begin, end)`	Returns part of the word	`"abcd".substring(1,3); // = "bc"`
`charAt(index)`	Returns the character at an index	`"abcd".charAt(2); // = "c"`
`trim()`	Removes whitespaces from both ends	`" Hello World ".trim(); // = "Hello World"`
`replace(search, add)`	Replaces part of the word	`"abcdefg".replace("cde", "klm"); // = "abklmfg"`
`equals(str)`	Check if string is equal to another string	`"abc".equals("abc"); // = true`
`equalsIgnoreCase(str)`	Equals with case ignored	`"abc".equalsIgnoreCase("AbC"); // = true`

Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	10 + 2	12
-	Subtraction	10 – 2	8
*	Multiplication	10 * 2	20
/	Division	10 / 2	5
%	Modulus (remainder)	10 % 2	0
++	Increment	a++ (consider a = 10)	11
--	Decrement	a-- (consider a = 10)	9
+=	Addition Assignment	a += 10 (consider a = 10)	20
-=	Subtraction assignment	a -= 10 (consider a = 10)	0
*=	Multiplication assignment	a *= 10 (consider a = 10)	100
/=	Division assignment	a /= 10 (consider a = 10)	1
%=	Modulus assignment	a %= 10 (consider a = 10)	0

Startertutorials.com

Arithmetic Operators

Post- vs Pre-

- Changes the way variables are saved
- Applies for both increment(++) and decrement(- -) operators

```
1 void example() {
2     int x = 10;
3     int y = x++;  // y = 10, x = 11
4     int z = ++x;  // z = 12, x = 12
5 }
6
```

```
y = x++ :
    y = x;
    x = x + 1;

z = ++x :
    x = x + 1;
    z = x;
```

```
y = x-- :
    y = x;
    x = x + 1;

z = --x :
    x = x + 1;
    z = x;
```

Logic Operators

OPERATOR	USAGE EXAMPLE	RETURNS “TRUE” VALUE IF…
>	a > b	**a** is greater than **b**
>=	a >= b	**a** is greater than or equal to **b**
<	a < b	**a** is less than **b**
<=	a <= b	**a** is less than or equal to **b**
==	a == b	**a** is equal to **b**
!=	a != b	**a** isn’t equal to **b**
&&	a && b	**a** and **b** are true values, **b** is evaluated relatively (if **a** is false, **b** isn’t evaluated)
||	a || b	**a** or **b** is true, **b** is evaluated relatively (if **a** is true, **b** isn’t evaluated)
!	!a	**a** is false
&	a & b	**a** and **b** are true, **b** is evaluated in any case
|	a | b	**a** or **b** is true, **b** is evaluated in any case

Logic Operators

&& vs &

```java
boolean a = false;
Boolean b = null;

boolean value = a && b;  // false

boolean exception = a & b;  // NullPointerException

```

- && will short-circuit if first value is `false`
- & will evaluate both values
- Same applies for the OR operand (| | vs |)

Exercise 1

What is the output of the given code?

```java
class OperatorExercise {
    public static void main(String[] args) {
        System.out.println(testOperators());
    }

    static boolean testOperators() {
        int x = 3;
        Integer y = null;
        boolean result;

        boolean flag = !(!false);

        int a = x++;
        int b = ++x;

        result = ((a < b) | (y != null && y > 2)) || flag;
        System.out.println("x = " + x + ", a = " + a + ", b = " + b);
        return result;
    }
}

```

If Statements

```java
String getValue(boolean a, boolean b) {
    if (a && b) {
        return "both are true";
    } else if(a && !b) {
        return "a true b false";
    } else {
        return "a false or b true";
    }
}

```

- Goes top → bottom
- Doesn't need all of the branches
- Always check for precedent!

If Statements

Structure

```
if (statement) {
    // Action
} else if (statement) {
    // Action
} else if (statement) {
    // Action
} else {
    // Action
}

```

If Statements

Inline

- The inline method for if/else is given by
  `boolean ? true : false`
- It can only cover two-branched if statements

```
// Only two branches
String getValue(boolean a, boolean b) {
    if (a && b) {
        return "both true";
    } else {
        return "i don't know";
    }
}

```

```
// Inline version
String getValue(boolean a, boolean b) {
    return (a && b) ? "both true" : "i don't know"
}

```

If Statements

Shorter

- Sometimes if statements don’t need explicit else statements
- However, use carefully

```
// Also legit
String getValue(boolean a, boolean b) {
    if (a && b) {
        return "both true";
    }
    return "i don’t know";
}

```

Switch Cases

```
enum Color { RED, YELLOW, GREEN }

boolean canIMove(Color color) {
    return switch(color) {
        case YELLOW, RED:
            false;
            break;
        case GREEN:
            true;
            break;
        default:
            true;
    }
}

```

- Skips to value
- Readible code
- Usually used together with `Enums`
- Allows multiple values

Switch Cases

Structure

```
switch(variable) {
    case value:
        // Action
        break;
    case value:
        // Action
        break;
    default:
        // Fallback action
}

```

Avoid

```
if (5 < 10) {
    // Always true
}
```

```
if (a > b) {
    if (a < b) {
        // Unreachable code
    }
}
```

- Constant expressions that are always `true/false`
- Using boolean expressions in `switch`

- Unreachable branches
- Unnecessary (`== true`) check

```
switch (a && b) {
    case true:
        // ...
}
```

```
if ((a && b) == true) {
    // ...
}
```

For Loops

- Repeat an action

```java
void printHello(int count) {
    for (int i=0; i<count; i++) {
        System.out.println("Hello to you sir");
    }
}

```

For Loops

Structure

```
for (init; condition; update) {
    // Action
}

```

For-Each Loops

- Iterate through elements

```
void iteratePeople(Person[] people) {
    for(Person person : people) {
        // Can reach each object (Person)
        person.action();
    }
}

```

For-Each Loops

Structure

```
Class[] multipleElements;
for(Class singleElement : multipleElements) {
    // Action
}

```

While Loops

```java
void printHello(int count) {
    int current = 0;
    while(current < count) {
        System.out.println("Hello to you sir");
        current++;
    }
}

```

- Similar to `for loop` structure
- Used mostly when ending unknown
- Can result in endless loops(careful)

While Loops

Structure

```
while(statement) {
    // Action
}

```

do-While Loops

```java
void printHello(int count) {
    int current = 0;
    do {
        System.out.println("Hello to you sir");
        current++;
    } while(current < count);
}

```

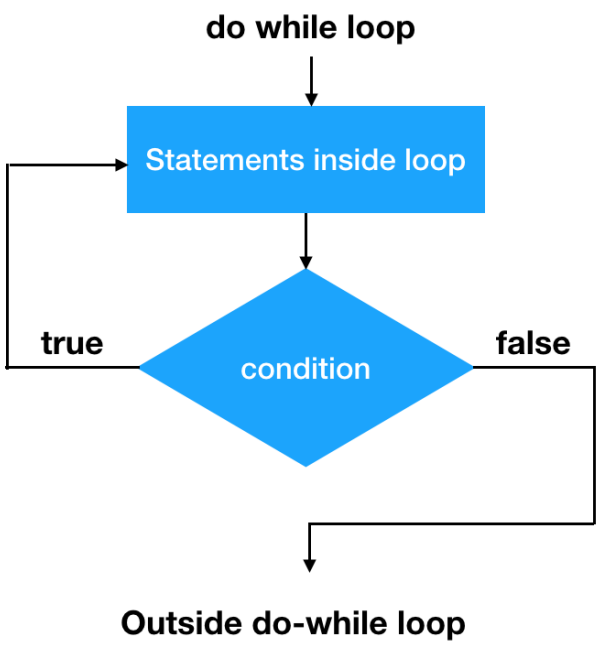
- Action done before checking statement
- Works once also when `count >= current`
- Required so rarely, can be mostly replaced with `for/while`

do-While Loops

Structure

```
do {
    // Action
} while(statement);

```



Break

```
1  void example() {
2      int i = 0;
3      while(true) {
4          if(i > 2) {
5              break;
6          }
7          // Do something
8      }
9  }
10
```

- Ends the whole loop
- When using `while loops` try avoiding `break` statements
- Can be used with any kind of loop

```
1  void example() {
2      for(int i=0; i<100; i++) {
3          if(i > 2) {
4              break;
5          }
6          // Do something
7      }
8  }
9
```

Reminder

- Break statements can also be found in switch cases

Continue

- Skips the current iteration
- Usually used with for loops

```java
void example() {
    for(int i=0; i<10; i++) {
        if (i == 2) {
            continue;
        }
        System.out.println(i);
        // 0, 1, 3, 4..
    }
}

```

Avoid

- Endless loops

```
while(true) {
    if (i > 10) {
        break;
    }
}
```

```
while(true) {
    // Endless loop
}
```

- Nested loop termination

Exercise 2

Rewrite the given do-While loop as a for loop

```
void printThis (int count, String sentence) {
    int i = 0;
    do {
        if (count == 1) {
            break;
        }
        System.out.println(sentence);
        i++;
    } while(i < count);
}

```

Exercise 3

- **Given:** `Tile[][] tiles`
- **Implement** the method:
  `void printRooks(Tile[][] tiles)`
- The method should iterate through the chess board
- For each ROOK on board: print the position of the piece

```
class Tile {
    int x;
    int y;
    Piece piece;
}

class Piece {
    Type type;
    // ...
}

enum Type {
    ROOK,
    BISHOP
    // ...
}
```

Conclusion

- Useful data type operators
- a++ vs ++a
- && vs &
- If statements
- Switch cases
- For loops
- While loops
- do-While loops
- Break & Continue

Remember

Important

Homework 1 deadline 20. Nov 06:00

Solution

Exercise 1

```
$ java OperatorExercise.java
x = 5, a = 3, b = 5
true
```

Solution

Exercise 2

```java
void printThis(int count, String sentence) {
    int i = 0;
    if (i >= count) {
        // We need to include 1 more iteration if expression false
        System.out.println(sentence);
    }
    for (; i < count; i++) {
        if (count == 1) {
            break;
        }
        System.out.println(sentence);
    }
}

```

Solution

Exercise 3

```java
void printRooks(Tile[][] tiles) {
    for (Tile[] row : tiles) {
        for (Tile tile : row) {
            Piece piece = tile.piece;
            if (piece.type == Type.ROOK) {
                int x = tile.x;
                int y = tile.y;
                System.out.println("x-position: " + x + ", y-position: " + y);
            }
        }
    }
}
```